

Firmware C++ y Hardware Electrónico de AetherNet: MEGA – Gateway ESP32 – Rover UNO con RF, EMA y Fail-safe

Andrés Felipe Martínez Henao

Firmware + Hardware – Electrónica Embebida

Tecnología en Desarrollo de Software (5º sem) – Ing. Sistemas y Computación (6º sem)

Universidad Tecnológica de Pereira

F.martinez5@utp.edu.co

Proyecto AetherNet – firmware/* + docs/Firmware + docs/Hardware
mega-access + gateway-esp32 + rover-uno – arduino-cli 1.5.1 – TT 6V 1:48

Abstract—Sin PDF UTP propio, Firmware/Hardware se evalúa como electrónica y sistemas embebidos trazables a RF-2.1/2.2/2.3/3.1/3.2 y HU-01.04 (RNF-1.2 CI). El Gateway ESP32-WROOM-32U + nRF24L01 (CE5/CSN15, SCK18/MOSI23/MISO19, Ch76 2 Mbps) puentea Wi-Fi/MQTT↔UART 38400 (RX16/TX17, divisor 5V→3.3V)↔RF. El MEGA 2560 integra keypad 4×4 (ROW 22/24/26/28, COL 30/32/34/36), servo MG90S pin 9 (0/90°), LED RGB ánodo común 44/45/46 PWM invertido, y láser KY-008 S→8 + LDR discreta 5V→•→7 +10k→GND INPUT (laser.cpp 50 ms CHECK, COOLDOWN 3 s, triggerIntrusionAlert→SECURITY:). El Rover UNO + L298N (ENA5/ENB11, IN1 6 IN2 7 IN3 8 IN4 9, drop 1.8V) + HC-SR04 TRIG2/ECHO3 con EMA $S_t = 0.2Y_t + 0.8S_{t-1} + 3 \times \text{TCRT5000 A0 trasero/A1 izq/A2 der threshold 500}$ ejecuta evasión y fail-safe 500 ms con checksum. El calce 2026-09-11 a TT 6V 1:48 (4 motores, 0.8 KG-cm, 120 RPM, ~0.35 m/s) impone -77% torque, 3.2A stall y MIN_PWM 60→70. Los 3 firmwares compilan en CI matriz ESP32/AVR (19252/6900 bytes) y el RF está validado HW 4/4 (testing-rf-sprint1.md:32, TX/RX L=120, isChipConnected=1). Pendientes: fotos 12 MP y telemetría TCRT/HC-SR04 validada e821700.

Index Terms—Arduino MEGA/UNO, ESP32, nRF24L01 RF24, HC-SR04 EMA $\alpha = 0.2$, TCRT5000, L298N, TT 1:48, KY-008 LDR, arduino-cli

I. INTRODUCCIÓN: FIRMWARE COMO CAPA EDGE

La capa Edge procesa sin depender de red (PRD §6 Contingencia): el cerrojo y el láser funcionan aunque caiga Wi-Fi/Docker. Firmware/Hardware comparten roadmap, pines y fritzing; su DoD es compilar en CI + banco físico. Referencias: docs/Firmware/firmware.md, roadmap-firmware.md, docs/Hardware/hardware.md, roadmap-hardware.md, hardware-inventory.md, fritzing/AetherNet-P*.png.

II. ARQUITECTURA HARDWARE

Fig. 1 (fritzing): P2 RF-Link, P3 MEGA-Cerrojo, P4 Rover, P5 Laser, P1 EMA-UNO – docs/fritzing/AetherNet-P2..P5*.png + compendio-planos.md.

Table I
MAPA COMPONENTE→FUNCIÓN→PINES (HARDWARE.MD §MAPA)

Componente	Función	Pines / protocolo
ESP32 + nRF24#1 MEGA + keypad + MG90S + LED	Gateway MW/MQTT↔UART↔RF Cerrojo HU-01 + feedback	CE5 CSN15 + UART16/17 38400 ROW 22/24/26/28 COL 30/32/34/36, S9, LED44/45/46
KY-008 + LDR 10k UNO + L298N + HC-SR04 + TCRT×3	Barrera HU-02 Rover HU-03/04	TX8, RX7 INPUT, 10k→GND ENA5 ENB11, TRIG2 ECHO3, A0/A1/A2
TT 6V 1:48 ×4	Chasis	2 motores/canal L298N

III. MÉTODO POR SUBSISTEMA

A. Gateway ESP32 (RF-2.1)

gateway-esp32.ino:59 RF24 Ch76 2 Mbps; UART 38400 a MEGA (RX16/TX17, divisor), MQTT a Mosquitto (aethernet/rover/command:69, access/command:70, access/event:71, seguridad/intrusion:72). Credenciales en secrets.h (gitignored) + secrets.h.example (DEVOPS-11). Traducción: MQTT↔UART JSON por línea + checksum, MQTT↔RF struct empaquetado #pragma pack.

B. MEGA Acceso/Potencia (RF-2.2/2.3, HU-01/02)

```
config.h: VALID_PIN 1234,
DOOR_AUTO_LOCK_MS 5000, LED_COMMON_ANODE
(255-valor), ROW/COL, SERVO_PIN 9, LASER_TX 8
LASER_RX 7 INPUT, DOOR_LOCKED 0 UNLOCKED
90. Lógica no bloqueante millis() (vs delay()):
laser.cpp:15 50 ms CHECK, led.cpp verde
5 s / rojo 1 s / azul 50 ms dígito, door.cpp +
keypad_control.cpp + uart_protocol.cpp:40
CMD:ACCESS 38400 bd. Flujo HU-01:
PIN→processPinAttempt==VALID_PIN→unlockDoor
90° 5s + LED verde + UART ACCESS:hash. HU-02:
laserInit/handleLaser/triggerIntrusionAlert
→ LED rojo 3 s + UART SECURITY:→Gateway→POST
/api/security-events +
```

Table II
CALCE ROVER (FUENTE HARDWARE-INVENTORY.MD §CALCE)

Parámetro	9–12V	TT 1:48	Impacto
Tensión	9–12V	6V (5V StepUp)	3.2V ef. → 2S 7.4V
Stall	0.4A	0.8A×4=3.2A	BEC 5V 3A + disp.
Vel	170–350 RPM	120 RPM 0.35 m/s	BASE 120→150
Torque	3.5 KG·cm	0.8 KG·cm	-77%, no rampa >15°
PWM min	60	70	recalibrar cargado

aethernet/seguridad/intrusion. Matriz relés eliminada 2026-08-26.

C. Rover UNO (RF-3.1/3.2, HU-03/04)

Tracción L298N (drop 1.8V vs TB6612FNG): ENA5/ENB11 PWM, IN1..4 6/7/8/9, 2 motores/canal. Evasión: HC-SR04 TRIG2 ECHO3 con EMA rover-uno.ino:259 $S_t = 0.2Y_t + 0.8S_{t-1}$ (init lazy, descarte fuera de rango, test-rover-sensors.ino 4712 b), umbrales 30/15 cm en executeAutoMode; anti-caída 3×TCRT5000 analogRead threshold 500 (A0 trasero, A1 izq, A2 der) – bug previo digitalRead(A0) corregido, invertido: blanca ~290 BORDE vs negra/vacío ~850 OK. RF: nRF24 RF24 lib CE4/CSN10, verifyChecksum:372–379, telemetría T_telemetry_invertida 89 (ir_left/right, ultrasonic 0–95, rf_rssi -70, mode). Fail-safe RF-3.3/HU-04: FAILSAFE_TIMEOUT_MS 500 (:80, 209–218) corta PWM si no llega paquete válido; no reintenta maniobra hasta nuevo comando.

D. Calce TT 6V 1:48 (2026-09-11)

Motivo: chasis oruga 9–12V no cabe en baúl lab → chasis TT compacto 4× TT 6V 1:48 (hardware-inventory.md:36, rover-uno.ino:6,89). Impacto Tab. II.

IV. RESULTADOS

Compilación CI: mega-access 19252 bytes/7%, rover-uno 6900 bytes/21% (arduino-cli 1.5.1, ArduinoJson 6.21.3, FQBN arduino:mega:atmega2560/arduino:avr:uno) – ci.yml:90. RF validado HW 2026-09-01 4/4 (ttyUSB0+ ttyACM0, RF TX/RX L=120, isChipConnected=1). Sprint 3 e821700: TCRT validado a 5 mm (A0 845 OK negra, 290 BORDE blanca), HC-SR04 US_raw 6–33 cm EMA 6–32 cm, OBSTACLE 30 cm, T_telemetry_invertida 89, joystick RF 250→249 99.6%, 33k/7.3k/1.5k telemetrías. Notebook Firmware_Notebook.ipynb 16 celdas + 20 logs docs/logs/firmware_sprint3/.

V. DISCUSIÓN

millis() no bloqueante es requisito para no congelar láser/TCRT; L298N drop y alimentación 2S son cuellos de botella del calce – TB6612FNG es alternativa documentada. El threshold TCRT invertido (blanca=borde) es contraintuitivo pero validado ópticamente y debe mantenerse. Pendiente: fotos

12 MP luz natural + regla (checklist README), video TCRT 5 mm + HC-SR04 EMA, y documentar MIN_PWM recalibrado en Firmware_Notebook.ipynb §4.

VI. CONCLUSIONES

El Edge AetherNet demuestra operación determinista con recursos limitados, EMA embebido idéntico al Python y fail-safe serie que satisface HU-04 sin añadir complejidad de Kalman.

REFERENCES

- [1] AetherNet, Documentación Firmware, docs/Firmware/firmware.md.
- [2] AetherNet, Roadmap Firmware, docs/Firmware/roadmap-firmware.md.
- [3] AetherNet, Documentación Hardware, docs/Hardware/hardware.md.
- [4] AetherNet, Roadmap Hardware, docs/Hardware/roadmap-hardware.md.
- [5] AetherNet, Hardware Inventory, docs/hardware-inventory.md (pines, calce TT).
- [6] AetherNet, firmware/rover-uno/rover-uno.ino (EMA:259, fail-safe:63, checksum:372).
- [7] AetherNet, firmware/mega-access/src/config.h + laser.cpp:15 + door.cpp + uart_protocol.cpp:40.
- [8] AetherNet, firmware/gateway-esp32/gateway-esp32.ino:59 (CE5/CSN15, Ch76).
- [9] AetherNet, docs/testing-rf-sprint1.md:32 (nRF24 4/4 validado).
- [10] AetherNet, docs/fritzing/AetherNet-P*.png + compendio-planos.md.